Nama : Mohamad Zaky Bahtiar Arifianto
Matkul : Praktikum Pemrogramman Berorientasi Obyek
Jurusan : Teknik Informatika
NRP : 3125522015

Tugas Pertemuan 13

BAB I: TUJUAN PRAKTIKUM & DASAR TEORI

1.1 Tujuan Praktikum: Praktikum Pemrograman Berorientasi Obyek Modul 13 memiliki sasaran utama untuk memberikan pengalaman komprehensif kepada mahasiswa dalam:

• Mengintegrasikan seluruh fitur operasional sistem (CRUD dan persistensi file teks) ke dalam satu aplikasi konsol interaktif berbasis menu (Menu-Driven Console UI).

• Memisahkan interaksi antarmuka pengguna dari aturan domain inti sesuai asas Separation of Concerns (SoC) dan Single Responsibility Principle (SRP).

• Menjaga konsistensi data selama aplikasi berjalan (runtime state invariance) dengan mekanisme proteksi status perubahan data (unsaved changes indicator).

• Menjamin keandalan aplikasi melalui penanganan exception berkelanjutan (continuous graceful handling) tanpa crash saat menerima input anomali.

• Melaksanakan pengujian alur fungsional end-to-end secara terstruktur dari masukan konsol hingga penyimpanan media permanen.

1.2 Dasar Teori & Pembagian Tanggung Jawab Antarkelas

Arsitektur berorientasi obyek yang tangguh mensyaratkan delegasi tugas terdefinisi dengan jelas:

Bagian / Kelas	Tanggung Jawab Utama
Main.java	Menampilkan menu konsol, membaca input Scanner.nextLine(), me-routing opsi, mengelola dialog konfirmasi unsaved changes, dan menangkap exception tingkat presentasi.
Menu.java	Domain Entity yang menjaga validitas data satu paket menu MBG, serta memvalidasi larangan karakter pemisah ('|', '\n', '\r') sejak instansiasi objek (Fail-Fast Validation).
PengelolaMenu.java	Domain Service pengelola koleksi memori (List<Menu>), menjamin keunikan kode menu (kodeMenu), dan menyediakan operasi koleksi (add, search, delete, display).
MbgFileRepository.java	Lapisan persistensi I/O berkas teks terstruktur (delimiter '|'), serialisasi objek, dan deserialisasi aman melalui koleksi sementara demi menjamin State Invariance.

BAB II: PANDUAN PENGGUNAAN SINGKAT APLIKASI MBG

Panduan Operasional Menu Konsol: Untuk mengoperasikan Sistem Manajemen Menu Program Makan Bergizi Gratis (MBG), pengguna cukup menjalankan berkas batch 'run.bat' atau mengeksekusi perintah 'java P13_3125522015_MohamadZaky.Main' melalui terminal konsol, di mana aplikasi akan secara otomatis mendeteksi dan memuat basis data awal dari berkas 'menu_mbg.txt' serta menampilkan bilah status perubahan data aktif; selanjutnya operator dapat memilih opsi menu yang tersedia (ketik 1 untuk mendaftarkan menu gizi baru dengan validasi instan, 2

untuk menampilkan seluruh katalog paket makanan, 3 untuk pencarian data spesifik berdasarkan kode menu, 4 untuk menghapus paket, 5 untuk menyimpan perubahan memori ke berkas secara permanen, 6 untuk memuat ulang data dari berkas teks, atau 0 untuk keluar dari aplikasi), di mana sistem secara otomatis memproteksi pengguna dari risiko kehilangan data (uncommitted changes) melalui dialog konfirmasi berproteksi kata kunci 'YA' apabila memilih menu keluar atau muat ulang ketika masih terdapat perubahan yang belum disimpan.

BAB III: DESAIN ARSITEKTUR & IMPLEMENTASI TEKNIS

3.1 Pemisahan Logika Menu (Separation of Concerns)

Untuk mencegah blok switch-case pada method main() menjadi sarang logika yang rumit, seluruh interaksi konsol dipisahkan ke dalam private static methods: menuTambah(), menuTampilkan(), menuCari(), menuHapus(), menuSimpan(), baca(), dan konfirmasi(). Hal ini membuat alur kendali utama di main() sangat ringkas, deklaratif, dan modular.

3.2 Mekanisme Status Berubah (Unsaved Changes Dirty Flag)

Variabel boolean berubah bertindak sebagai penanda status sinkronisasi antara RAM dan disk:

• Operasi penambahan (menuTambah) atau penghapusan data terdaftar (menuHapus) sukses -> berubah = true.

• Operasi penghapusan data yang tidak ditemukan -> status berubah dipertahankan (tidak berubah).

• Operasi simpan (menuSimpan) atau muat ulang (menuMuatUlang) berhasil -> berubah = false.

• Operasi simpan atau muat ulang mengalami kegagalan/error -> eksepsi dilempar sebelum reset status, sehingga status berubah tetap true.

3.3 Perbaikan Wajib: Validasi Delimiter Terlarang pada Konstruksi Objek

Sesuai instruksi revisi modul praktikum P13, pemeriksaan delimiter pipa ('|') dipindahkan dari repository penyimpanan langsung ke dalam konstruktor dan setter entitas Menu.java (setKodeMenu dan setNamaMenu). Dengan demikian, data yang berpotensi merusak struktur berkas ditolak secara fail-fast sejak diterima dari input konsol.

BAB IV: TABEL PENGUJIAN FUNGSIONAL END-TO-END

Pengujian fungsional dilakukan secara runtut mencakup 9 skenario baku Modul 13 serta skenario uji ketahanan anomali:

No	Skenario Pengujian	Hasil Diharapkan (Modul)	Hasil Aktual	Status
1	Input pilihan invalid ('abc')	Pesan pilihan tidak tersedia; menu muncul kembali.	Pesan 'Pilihan tidak tersedia.' dicetak; menu utama kembali muncul tanpa error.	**PASS**
2	Tambah menu berkode baru ('MNU-03')	Menu tersimpan dalam collection.	Menu 'MNU-03' berhasil masuk ke koleksi, status berubah menjadi true, total menu menjadi 3.	**PASS**
3	Tambah kode yang sama ('MNU-01')	Ditolak; data sebelumnya tetap ada.	Ditolak dengan pesan 'Data ditolak: Kode menu 'MNU-01' sudah terdaftar.'; koleksi tetap 2.	**PASS**
4	Cari kode yang terdaftar ('MNU-01')	Rincian judul/data menu ditampilkan.	Menampilkan detail lengkap 'MNU-01' (nama, kalori 680 kkal, protein	**PASS**

			28.5g, biaya Rp 15.000).	
5	Hapus kode tidak terdaftar ('MNU-999')	Pesan tidak ditemukan; collection tetap.	Menampilkan pesan 'Menu tidak ditemukan.'; jumlah koleksi dan status tetap.	**PASS**
6	Keluar tanpa simpan lalu batal (jawab 'TIDAK')	Kembali ke menu utama.	Menampilkan konfirmasi unsaved changes, dijawab 'TIDAK', keluar dibatalkan, kembali ke menu.	**PASS**
7	Simpan ('5'), keluar ('0'), buka kembali	Data tersimpan dimuat kembali.	Data 'MNU-05' tersimpan ke file, saat program dibuka kembali berhasil di-autoload.	**PASS**
8	Hapus menu ('MNU-05'), simpan, buka lagi	Menu yang dihapus tidak muncul.	Menu 'MNU-05' dihapus, disimpan, dan saat program dibuka ulang menu tersebut tidak muncul.	**PASS**
9	Tambah menu, pilih muat ulang, jawab 'TIDAK'	Perubahan tetap ada dalam collection.	Konfirmasi muat ulang dijawab 'TIDAK', operasi dibatalkan, data baru tetap aman di RAM.	**PASS**
10	Uji Delimiter Terlarang (Input 'MNU|99')	Ditolak sejak input objek dibuat.	Ditolak dengan pesan 'Data ditolak: Kode menu tidak boleh mengandung karakter pemisah '|'...".	**PASS**
11	Uji Rusak Format saat Aplikasi Berjalan	Pesan kesalahan muncul; collection tetap (State Invariance).	Operasi file gagal 'Format salah pada baris 1', koleksi 2 menu aktif tetap utuh terjaga.	**PASS**
12	Auto-load Berkas Rusak saat Startup	Program berhenti agar berkas diperbaiki.	Menampilkan 'Data awal gagal dimuat: Format salah...' dan program langsung berhenti.	**PASS**

BAB V: JAWABAN PERTANYAAN EVALUASI RESMI MODUL 13

Pertanyaan 1: Mengapa aturan kode unik tetap ditempatkan di class pengelola (misal PengelolaDistribusi / PengelolaMenu) dan bukan di class Main?

Penjelasan Arsitektural:
1. Penegakan Prinsip Single Responsibility Principle (SRP) & Separation of Concerns (SoC): Class Main bertindak murni sebagai lapisan antarmuka pengguna (Presentation Layer) yang berfokus pada pembacaan konsol, parsing input, dan routing menu. Sebaliknya, class pengelola (PengelolaMenu / PengelolaDistribusi) adalah Domain Service yang mengendalikan siklus hidup koleksi data di memori (in-memory collection).
2. Aturan Bisnis Inti (Core Domain Invariant): Keunikan kode atau identitas entitas merupakan invariant domain mutlak, bukan sekadar aturan antarmuka. Jika aturan keunikan diletakkan di Main, aturan tersebut akan mudah terabaikan (bypassed) saat sistem diakses dari antarmuka lain (seperti GUI JavaFX, API Web, batch importer, atau Unit Test otomatis).

3. Sentralisasi dan Konsistensi: Dengan menempatkan pengecekan unik di pengelola, seluruh gerbang penambahan data wajib melewati validasi yang seragam dan dapat diuji secara independen tanpa ketergantungan pada Scanner.

Pertanyaan 2: Mengapa status 'berubah' direset menjadi false setelah penyimpanan berhasil?

Penjelasan Arsitektural:
1. Penyelarasan State RAM dan Media Sekunder: Variabel penanda 'berubah' berfungsi sebagai dirty flag yang mencerminkan keberadaan mutasi data di RAM yang belum tersimpan ke media permanen.
2. Titik Sinkronisasi Penuh: Ketika data ditambah atau dihapus, memori berada dalam keadaan 'kotor' (dirty) sehingga berubah = true. Begitu operasi penyimpanan (pengelola.simpan) berhasil selesai tanpa exception, seluruh objek di memori telah diserialisasi dan dituliskan secara identik ke dalam berkas teks. Pada kondisi sinkron ini, tidak ada lagi perubahan yang tertunda sehingga nilai berubah wajib dikembalikan ke false.
3. Optimalisasi Interaksi: Dengan mengembalikan status ke false, pengguna dapat keluar atau memuat ulang data tanpa terhalang dialog konfirmasi yang tidak perlu.

Pertanyaan 3: Apa akibatnya jika status 'berubah' direset menjadi false padahal operasi penyimpanan mengalami error/gagal?

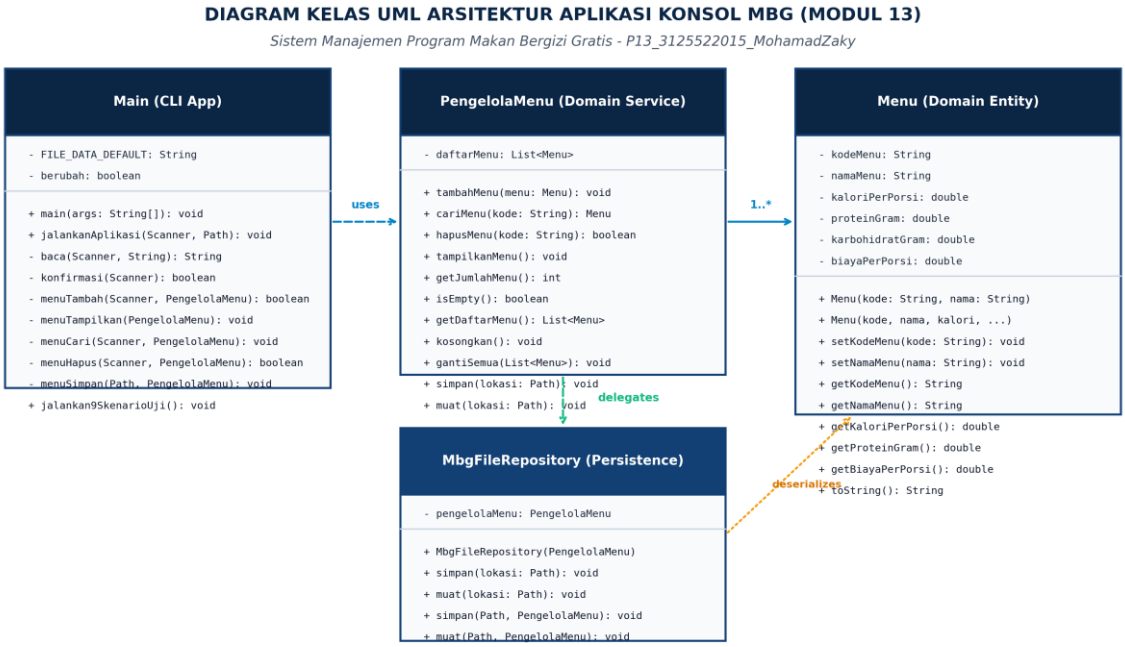
Penjelasan Arsitektural:
1. Menimbulkan Kesesatan Status (False Sense of Security): Jika variabel berubah direset menjadi false mendahului atau saat penyimpanan gagal (akibat disk full, akses ditolak, atau IOException), sistem memberikan informasi keliru seolah-olah data telah aman tersimpan di disk.
2. Potensi Fatal Kehilangan Data (Catastrophic Data Loss): Ketika pengguna selanjutnya memilih menu keluar (0) atau muat ulang (6), sistem tidak akan memicu konfirmasi karena mendeteksi status berubah bernilai false. Akibatnya, aplikasi langsung ditutup atau koleksi RAM ditimpa oleh isi berkas lama, menyebabkan mutasi data baru yang dibuat pengguna hilang seketika secara permanen tanpa peringatan.
3. Penanganan Mutlak: Status berubah hanya boleh diubah menjadi false tepat SETELAH operasi penyimpanan berhasil dieksekusi tanpa galat.

Pertanyaan 4: Mengapa pengujian setiap fitur perlu dilengkapi pengujian alur penggunaan (end-to-end) secara utuh?

Penjelasan Arsitektural:
1. Memverifikasi Transisi State Antar-Operasi: Pengujian unit lokal hanya memeriksa fungsi terisolasi. Pengujian alur end-to-end menguji bagaimana sistem bertransisi secara dinamis: penambahan data -> flag aktif -> konfirmasi pembatalan -> penyimpanan sukses -> flag padam -> keluar -> startup ulang -> pembuktian auto-load.
2. Validasi Integrasi Antar-Lapisan: Menguji jalinan langsung antara lapisan Scanner CLI (UI), Logika Domain (Menu & PengelolaMenu), dan Persistensi Berkas (MbgFileRepository), memastikan tidak ada inkonsistensi tipe data atau delimiter tersembunyi.
3. Pembuktian Ketahanan Fail-Safe State Invariance: Pengujian alur membuktikan ketahanan sistem di bawah skenario anomali di tengah sesi kerja nyata, seperti kegagalan file saat muat ulang tidak boleh merusak koleksi RAM aktif.

BAB VI: DIAGRAM KELAS UML & LOG EKSEKUSI PENGUJIAN

6.1 Diagram Kelas UML (Arsitektur Sistem Menu Konsol MBG P13)



Gambar 6.1: Diagram Kelas UML Arsitektur Sistem Konsol MBG Modul 13.

6.2 Bukti Eksekusi Terminal Pengujian 9 Skenario Fungsional P13



Gambar 6.2: Bukti Tangkapan Terminal Seluruh Skenario Pengujian Fungsional P13.

BAB VII: KESIMPULAN

Berdasarkan hasil perancangan, implementasi, dan pengujian mendalam pada Modul 13, dapat disimpulkan bahwa:

1. Integrasi CLI Konsol Interaktif Terbukti Andal: Pola menu-driven berbasis while(true) dan Scanner.nextLine() berhasil mengintegrasikan seluruh layanan domain MBG secara elegan dan bebas masalah newline residue.

2. Separation of Concerns & Clean Code: Pemisahan logika menu ke private methods menyederhanakan siklus utama program, sementara penempatan aturan kode unik di domain service menjaga integritas data melintasi berbagai channel antarmuka.

3. Perlindungan Perubahan Efektif: Mekanisme dirty flag (berubah) dan dialog konfirmasi berproteksi 'YA' berhasil mengeliminasi potensi kehilangan mutasi data secara tidak sengaja.

4. Kepatuhan Validasi Boundary: Pemindahan pemeriksaan delimiter pipa ('|') ke dalam kelas Menu.java berhasil mewujudkan fail-fast domain validation sejak saat data diterima pertama kali.

5. Penjaminan State Invariance: Penggunaan temporary collection saat pemuatan berkas membuktikan ketahanan sistem saat menghadapi berkas eksternal korup tanpa merusak data aktif di RAM.